# Using Copilot + Automated Scanning to Detect Insecure Code

**Submitted By:** Suraj Somkuwar

**GitHub Project Repository:** https://github.com/surajs-sudo/copilot-security-lab

# 1. Copilot-Generated Insecure Code

GitHub Copilot was used to generate an initial login implementation for security testing. The generated code contained insecure patterns that required security validation.

**Screenshot:  AI-Generated Insecure Login Source Code**

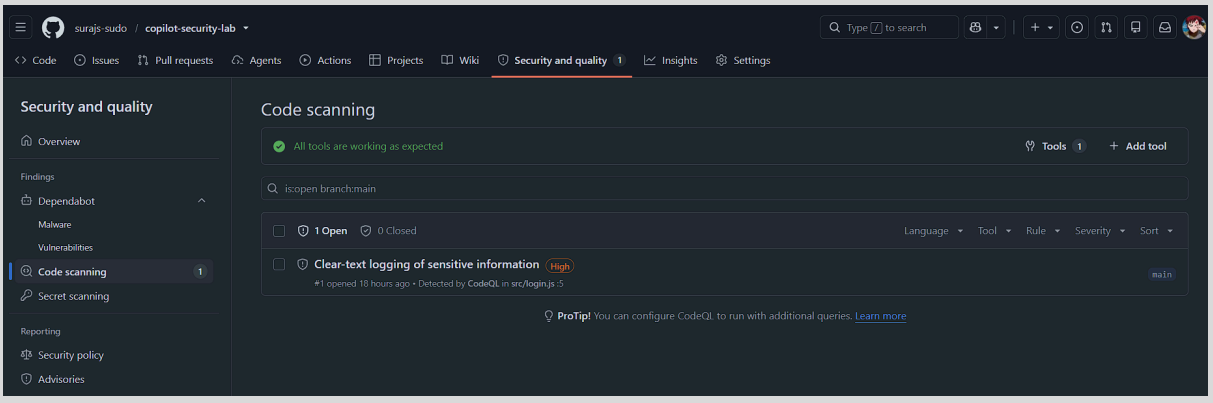

# 2. Code Scanning Alert Detection

The vulnerable implementation was pushed to GitHub and automatically analyzed using GitHub CodeQL.

**CodeQL detected:**

*Clear-text logging of sensitive information*

**Screenshot: GitHub Security and Quality Code Scanning Result/Alert**

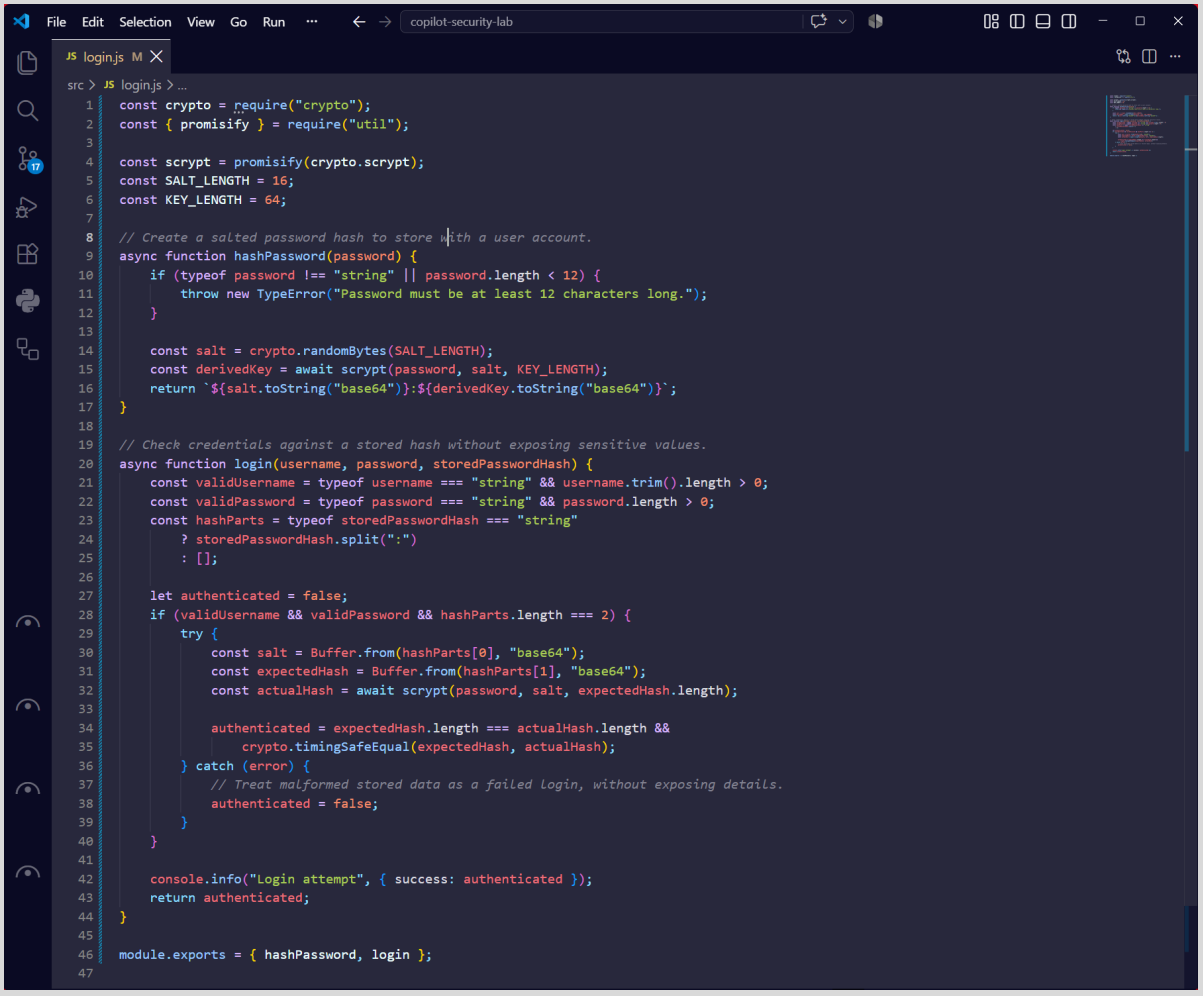


# 3. Secure Code Remediation

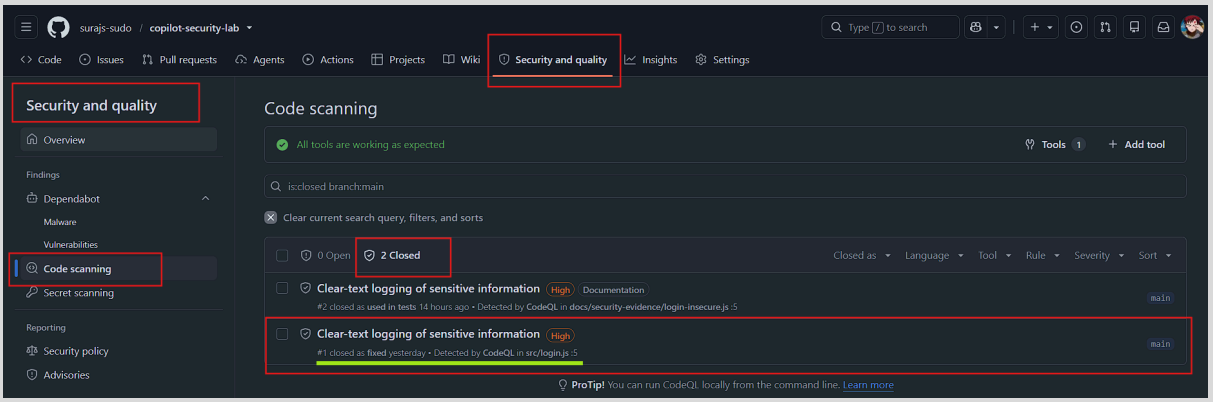
After reviewing the security finding, GitHub Copilot was used to rewrite the implementation securely.

The vulnerable logging behavior was removed and the updated code was scanned again.

### Screenshot: Final Secure Login Source Code Implementation




### Screenshot: CodeQL Closed Alerts Showing Fixed Vulnerability



# 4. Security Analysis Summary

## What insecure patterns did Copilot generate, and how did scanning help catch them?

*GitHub Copilot was used to generate an initial login authentication function for security testing. Although AI coding assistants can improve developer productivity, the generated code must always be reviewed and tested because AI-generated code may contain insecure implementation patterns.*

*The initial Copilot-generated login implementation contained multiple security weaknesses.*

## Analysis of AI-Generated Insecure Code

The original insecure implementation contained the following code behavior:

***function login(username, password) {***

This function accepts a username and password directly without applying any security controls.

## Security Issue:

The function processes sensitive authentication information without protection.

A secure authentication system should avoid handling passwords in plain text whenever possible.

## 1. Exposure of Sensitive Information Through Logging

The insecure code contained:

***console.log("Username:", username);***

***console.log("Password:", password);***

**What does this mean?**

The application was writing the username and password values directly into application logs.

For example, if a user entered:

***Username: admin***

***Password: password123***

the application log could store:

***Username: admin***

***Password: password123***

**Why is this insecure?**

Application logs are often stored, monitored, or accessed by different systems and administrators.

If attackers gain access to logs, they may discover sensitive authentication information.

This creates a risk of:

- Credential exposure.
- Unauthorized account access.
- Sensitive information leakage.

This was the primary vulnerability detected by GitHub CodeQL:

***Clear-text logging of sensitive information***

## 2. Hardcoded Authentication Credentials

The insecure code also contained:

***return username === "admin" && password === "password123";***

**What does this mean?**

The application directly compares user input against fixed credentials written inside the source code.

The username:

***admin***

and password:

***password123***

are permanently stored in the application.

**Why is this insecure?**

Hardcoded credentials create several security risks:

- Anyone with access to the source code can discover the credentials.
- Password changes require modifying application code.
- The same credentials may accidentally be reused in production.
- Attackers can easily guess weak passwords.

## 3. Missing Password Protection

The insecure implementation compares passwords directly:

***password === "password123"***

**Security Issue:**

Passwords should never be stored or compared as plain text.

Secure applications normally use:

- Password hashing.
- Salt generation.
- Secure comparison methods.

The insecure implementation did not provide these protections.

## How GitHub CodeQL Helped Detect the Issues

After the vulnerable code was pushed to GitHub, GitHub CodeQL automatically analyzed the repository through GitHub Actions.

**The scanning process:**

***Code Push***

↓

***GitHub Actions Triggered***

↓

***CodeQL Source Analysis***

↓

***Security Pattern Detection***

↓

***Security Alert Generated***

**CodeQL identified the insecure behavior and generated a security alert under:**

***GitHub Security and Quality***

↓

***Code Scanning Alerts***

**The alert provided:**

- Vulnerable file location.
- Problematic code section.
- Security explanation.
- Recommended remediation guidance.

This helped identify that the application was exposing sensitive information through insecure logging.

## AI-Assisted Secure Remediation

- After reviewing the CodeQL findings, GitHub Copilot was used again to generate a secure implementation.
- The insecure code was replaced with a secure version.
- The improved implementation introduced several security improvements.

## 1. Password Hashing Instead of Plain Text Storage

The secure implementation uses cryptographic hashing:

```
const scrypt = promisify(crypto.scrypt);
```

The password is converted into a secure hash before storage.

The implementation also creates a random salt:

```
const salt = crypto.randomBytes(SALT_LENGTH);
```

**Security Improvement:**

Even if stored password data is exposed, attackers cannot directly retrieve the original password.

## 2. Password Length Validation

The secure code validates password input:

```
if (typeof password !== "string" || password.length < 12)
```

**Security Improvement:**

This prevents weak passwords from being accepted and improves authentication security.

## 3. Removal of Sensitive Data Logging

The insecure implementation logged:

*console.log("Password:", password);*

The secure implementation removed password exposure.

Instead, it logs only the authentication result:

*console.info("Login attempt", { success: authenticated });*

**Security Improvement:**

Logs no longer contain sensitive authentication information.

## 4. Secure Password Comparison

The secure implementation uses:

*crypto.timingSafeEqual()*

**Security Improvement:**

This provides safer comparison of password hashes and reduces the risk of timing-based attacks.

## Final Security Validation

After applying the secure remediation:

1. The insecure implementation was replaced.
2. The updated code was pushed to GitHub.
3. CodeQL automatically scanned the repository again.
4. The previous vulnerability was marked as fixed.

**The final result demonstrated:**

*AI Generated Code*

↓

*Security Review*

↓

*CodeQL Detection*

↓

*AI-Assisted Fix*

↓

*Automated Validation*

## Conclusion

This exercise demonstrated that AI-generated code should not be trusted without security verification.

GitHub Copilot successfully accelerated development, but the generated insecure implementation introduced risks such as:

- Clear-text logging of sensitive information.
- Hardcoded credentials.
- Plain-text password comparison.

GitHub CodeQL provided an automated security validation layer that detected these weaknesses and guided remediation.

The combination of AI assistance and automated security scanning demonstrates an effective DevSecOps approach:

**AI helps create and improve code, while security tools verify that the code is safe before deployment.**